

Ejemplos sencillos en diferentes BD

Nota: recuerden que son ejemplos muy básicos a los que deben mejorar y agregar controles según la solución que están construyendo.

Algunas consideraciones conversadas en clase:

Agregar como parámetro en los diferentes métodos variables de control de error.

Ver inclusión de código hash para validar certificados válidos en la operación.

Realizar la inclusión de estas pruebas en sus trabajos de campo, en el motor de BD Y herramientas de desarrollo que prefieran.

Recordar que para poder hacer una trazabilidad de su modelo de clases al mapeo en la bd relacional, si el motor elegido no contempla creación de paquetes puedo sustituir identificando en el nombre de los procedimientos o funciones su dependencia. Ejemplo: Per_AgregarPersona(X1,X2,X3,):y,z, indicando al método que corresponde a la clase persona.

BD Oracle

Creación de tabla

```
CREATE TABLE Persona (  
    id_persona NUMBER PRIMARY KEY,  
    nombre VARCHAR2(100),  
    apellido VARCHAR2(100),  
    edad NUMBER
```

);

—Creación de paquete de BD para administrar la tabla persona.

```
CREATE OR REPLACE PACKAGE PersonaPackage  
AS
```

```
-- Procedimiento para insertar una nueva persona
```

```
PROCEDURE insertarPersona(  
  p_nombre IN VARCHAR2,  
  p_apellido IN VARCHAR2,  
  p_edad IN NUMBER  
);
```

```
-- Procedimiento para actualizar el nombre de una  
persona
```

```
PROCEDURE actualizarNombre(  
  p_id_persona IN NUMBER,  
  p_nuevo_nombre IN VARCHAR2  
);
```

```
-- Procedimiento para actualizar el apellido de una  
persona
```

```
PROCEDURE actualizarApellido(  
  p_id_persona IN NUMBER,  
  p_nuevo_apellido IN VARCHAR2  
);
```

-- Procedimiento para actualizar la edad de una persona

```
PROCEDURE actualizarEdad(  
    p_id_persona IN NUMBER,  
    p_nueva_edad IN NUMBER  
);
```

-- Función para obtener el nombre de una persona

```
FUNCTION obtenerNombre(  
    p_id_persona IN NUMBER  
) RETURN VARCHAR2;
```

-- Función para obtener el apellido de una persona

```
FUNCTION obtenerApellido(  
    p_id_persona IN NUMBER  
) RETURN VARCHAR2;
```

-- Función para obtener la edad de una persona

```
FUNCTION obtenerEdad(  
    p_id_persona IN NUMBER  
) RETURN NUMBER;
```

```
END PersonaPackage;
```

```
/
```

```
CREATE OR REPLACE PACKAGE BODY  
PersonaPackage AS
```

-- Procedimiento para insertar una nueva persona

```
PROCEDURE insertarPersona(  
    p_nombre IN VARCHAR2,  
    p_apellido IN VARCHAR2,  
    p_edad IN NUMBER  
) AS  
BEGIN  
    INSERT INTO Persona (id_persona, nombre,  
apellido, edad)  
    VALUES (SEQ_PERSONA.NEXTVAL, p_nombre,  
p_apellido, p_edad);  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Error al  
insertar persona: ' || SQLERRM);  
END insertarPersona;
```

-- Procedimiento para actualizar el nombre de una persona

```
PROCEDURE actualizarNombre(  
    p_id_persona IN NUMBER,  
    p_nuevo_nombre IN VARCHAR2  
) AS  
BEGIN  
    UPDATE Persona  
    SET nombre = p_nuevo_nombre  
    WHERE id_persona = p_id_persona;  
    IF SQL%NOTFOUND THEN  
        RAISE_APPLICATION_ERROR(-20002, 'No se  
encontró la persona con ID: ' || p_id_persona);
```

```
END IF;  
EXCEPTION  
WHEN OTHERS THEN  
    RAISE_APPLICATION_ERROR(-20003, 'Error al  
actualizar nombre: ' || SQLERRM);  
END actualizarNombre;
```

-- Procedimiento para actualizar el apellido de una
persona

```
PROCEDURE actualizarApellido(  
    p_id_persona IN NUMBER,  
    p_nuevo_apellido IN VARCHAR2  
) AS  
BEGIN  
    UPDATE Persona  
    SET apellido = p_nuevo_apellido  
    WHERE id_persona = p_id_persona;  
    IF SQL%NOTFOUND THEN  
        RAISE_APPLICATION_ERROR(-20004, 'No se  
encontró la persona con ID: ' || p_id_persona);  
    END IF;  
EXCEPTION  
WHEN OTHERS THEN  
    RAISE_APPLICATION_ERROR(-20005, 'Error al  
actualizar apellido: ' || SQLERRM);  
END actualizarApellido;
```

-- Procedimiento para actualizar la edad de una
persona

```
PROCEDURE actualizarEdad(
  p_id_persona IN NUMBER,
  p_nueva_edad IN NUMBER
) AS
BEGIN
  UPDATE Persona
  SET edad = p_nueva_edad
  WHERE id_persona = p_id_persona;
  IF SQL%NOTFOUND THEN
    RAISE_APPLICATION_ERROR(-20006, 'No se
encontró la persona con ID: ' || p_id_persona);
  END IF;
EXCEPTION
  WHEN OTHERS THEN
    RAISE_APPLICATION_ERROR(-20007, 'Error al
actualizar edad: ' || SQLERRM);
END actualizarEdad;

-- Función para obtener el nombre de una persona
FUNCTION obtenerNombre(
  p_id_persona IN NUMBER
) RETURN VARCHAR2 AS
  v_nombre Persona.nombre%TYPE;
BEGIN
  SELECT nombre INTO v_nombre
  FROM Persona
  WHERE id_persona = p_id_persona;
  RETURN v_nombre;
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20008, 'Error al
obtener nombre: ' || SQLERRM);
END obtenerNombre;
```

-- Función para obtener el apellido de una persona

```
FUNCTION obtenerApellido(
    p_id_persona IN NUMBER
) RETURN VARCHAR2 AS
    v_apellido Persona.apellido%TYPE;
BEGIN
    SELECT apellido INTO v_apellido
    FROM Persona
    WHERE id_persona = p_id_persona;
    RETURN v_apellido;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20009, 'Error al
obtener apellido: ' || SQLERRM);
END obtenerApellido;
```

-- Función para obtener la edad de una persona

```
FUNCTION obtenerEdad(
    p_id_persona IN NUMBER
) RETURN NUMBER AS
```

```
v_edad Persona.edad%TYPE;
BEGIN
  SELECT edad INTO v_edad
  FROM Persona
  WHERE id_persona = p_id_persona;
  RETURN v_edad;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
  WHEN OTHERS THEN
    RAISE_APPLICATION_ERROR(-20010, 'Error al
obtener edad: ' || SQLERRM);
  END obtenerEdad;

END PersonaPackage;
/
```

BD MySql

Crear una tabla

```
CREATE TABLE Persona (
  id_persona INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  apellido VARCHAR(100),
  edad INT
```

);

Crear Procedimientos y Funciones, para administrar la tabla del ejemplo.

DELIMITER //

```
CREATE PROCEDURE insertarPersona(  
    p_nombre VARCHAR(100),  
    p_apellido VARCHAR(100),  
    p_edad INT  
)  
BEGIN  
    INSERT INTO Persona (nombre, apellido, edad)  
    VALUES (p_nombre, p_apellido, p_edad);  
END//
```

```
CREATE PROCEDURE actualizarNombre(  
    p_id_persona INT,  
    p_nuevo_nombre VARCHAR(100)  
)  
BEGIN  
    UPDATE Persona  
    SET nombre = p_nuevo_nombre  
    WHERE id_persona = p_id_persona;  
END//
```

```
CREATE PROCEDURE actualizarApellido(  
    p_id_persona INT,  
    p_nuevo_apellido VARCHAR(100)  
)  
BEGIN  
    UPDATE Persona  
    SET apellido = p_nuevo_apellido  
    WHERE id_persona = p_id_persona;  
END//
```

```
    p_id_persona INT,  
    p_nuevo_apellido VARCHAR(100)  
)  
BEGIN  
    UPDATE Persona  
    SET apellido = p_nuevo_apellido  
    WHERE id_persona = p_id_persona;  
END//
```

```
CREATE PROCEDURE actualizarEdad(  
    p_id_persona INT,  
    p_nueva_edad INT  
)  
BEGIN  
    UPDATE Persona  
    SET edad = p_nueva_edad  
    WHERE id_persona = p_id_persona;  
END//
```

```
CREATE FUNCTION obtenerNombre(p_id_persona  
INT) RETURNS VARCHAR(100)  
BEGIN  
    DECLARE v_nombre VARCHAR(100);  
    SELECT nombre INTO v_nombre  
    FROM Persona  
    WHERE id_persona = p_id_persona;  
    RETURN v_nombre;  
END//
```

```
CREATE FUNCTION obtenerApellido(p_id_persona
INT) RETURNS VARCHAR(100)
BEGIN
    DECLARE v_apellido VARCHAR(100);
    SELECT apellido INTO v_apellido
    FROM Persona
    WHERE id_persona = p_id_persona;
    RETURN v_apellido;
END//
```

```
CREATE FUNCTION obtenerEdad(p_id_persona INT)
RETURNS INT
BEGIN
    DECLARE v_edad INT;
    SELECT edad INTO v_edad
    FROM Persona
    WHERE id_persona = p_id_persona;
    RETURN v_edad;
END//
```

DELIMITER ;

Guardar valores anteriores de un atributo al modificar

Manejo de triggers en MySql-----

Para actualizar el nombre de una persona en la tabla y
llevar el valor anterior a un histórico.

```
CREATE TABLE PersonaHistorica (  
    id_persona INT,  
    nombre_antiguo VARCHAR(100),  
    nombre_nuevo VARCHAR(100),  
    fecha_actualizacion TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

```
DELIMITER //
```

```
CREATE TRIGGER actualizar_nombre_persona_trigger  
AFTER UPDATE ON Persona  
FOR EACH ROW  
BEGIN  
    IF OLD.nombre != NEW.nombre THEN  
        INSERT INTO PersonaHistorica (id_persona,  
nombre_antiguo, nombre_nuevo)  
VALUES (OLD.id_persona, OLD.nombre,  
NEW.nombre);  
    END IF;  
END//
```

```
DELIMITER ;
```

```
-----  
en BD ORACLE-----  
-----
```

```
CREATE TABLE PersonaHistorica (
```

```
id_persona NUMBER,  
nombre_antiguo VARCHAR2(100),  
nombre_nuevo VARCHAR2(100),  
fecha_actualizacion TIMESTAMP  
);
```

```
CREATE OR REPLACE TRIGGER  
actualizar_nombre_persona_trigger  
BEFORE UPDATE OF nombre ON Persona  
FOR EACH ROW  
BEGIN  
    IF :OLD.nombre != :NEW.nombre THEN  
        INSERT INTO PersonaHistorica (id_persona,  
nombre_antiguo, nombre_nuevo, fecha_actualizacion)  
VALUES (:OLD.id_persona, :OLD.nombre,  
:NEW.nombre, SYSTIMESTAMP);  
    END IF;  
END;  
/
```

Crear desde una función en MySql una api en Java

```
DELIMITER //
```

```
CREATE FUNCTION actualizar_nombre_persona(id  
INT, nuevo_nombre VARCHAR(100)) RETURNS INT  
BEGIN
```

```
UPDATE Persona SET nombre = nuevo_nombre
WHERE id_persona = id;
RETURN ROW_COUNT();
END//
```

DELIMITER ;

Herramientas utilizadas para este ejemplo:

Spring Boot y Maven para manejar las dependencias.

Primero, necesitas crear un proyecto Spring Boot. Puedes hacerlo desde Spring Initializer o mediante tu IDE favorito.

Agrega las dependencias necesarias en tu archivo pom.xml. Para este ejemplo, necesitarás la dependencia para MySQL Connector/J para conectarte a la base de datos MySQL.

```
<dependencies>
  <!-- Spring Boot Starter Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- MySQL Connector/J -->
  <dependency>
```

```
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
</dependency>
</dependencies>
```

Crea un controlador en Java que maneje las solicitudes HTTP para actualizar el nombre de la persona. En este controlador, inyectaremos un JdbcTemplate para ejecutar la función de MySQL.

```
import
org.springframework.beans.factory.annotation.Autowired
;
import org.springframework.jdbc.core.JdbcTemplate;
import
org.springframework.web.bind.annotation.PathVariable;
import
org.springframework.web.bind.annotation.PutMapping;
import
org.springframework.web.bind.annotation.RequestMapping;
;
import
org.springframework.web.bind.annotation.RestController
;
```

```
@RestController
@RequestMapping("/api/persona")
public class PersonaController {
```

```
@Autowired
private JdbcTemplate jdbcTemplate;

@PutMapping("/{id}/nombre/{nuevoNombre}")
public String actualizarNombre(@PathVariable int id,
@PathVariable String nuevoNombre) {
    String sql = "SELECT
actualizar_nombre_persona(?, ?)";
    int filasActualizadas = jdbcTemplate.update(sql, id,
nuevoNombre);
    if (filasActualizadas > 0) {
        return "Nombre actualizado correctamente";
    } else {
        return "No se pudo actualizar el nombre";
    }
}
}
```

En este controlador, estamos utilizando `@PutMapping` para manejar las solicitudes PUT HTTP en la URL `/api/persona/{id}/nombre/{nuevoNombre}`. Inyectamos un `JdbcTemplate` para ejecutar la función de MySQL que actualiza el nombre de la persona.

Configura la conexión a tu base de datos MySQL en el archivo `application.properties`.

```
spring.datasource.url=jdbc:mysql://localhost:3306/tu_ba
sededatos
spring.datasource.username=tu_usuario
spring.datasource.password=tu_contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.
Driver
```

Con esto, has creado una API en Java que utiliza una función de MySQL para realizar el cambio de nombre de una persona en una tabla "Persona". Asegúrate de tener configurada correctamente tu base de datos y ajusta el código según sea necesario para tu aplicación.

-----Comparamos el ejemplo visto en clase sobre alumnos que se inscriben a cursos y cómo sería su diseño en una base de datos no relacional.

BD MongoDB- almacenar información sobre la inscripción de alumnos a cursos

```
{
  "curso": {
    "_id": "curso_id",
    "nombre": "Nombre del Curso",
    "descripcion": "Descripción del Curso",
    "fecha_inicio": "2024-04-15",
    "fecha_fin": "2024-05-15",
```

```
"cupos_disponibles": 20,  
"alumnos_inscritos": [  
  {  
    "_id": "alumno_id",  
    "nombre": "Nombre del Alumno",  
    "apellido": "Apellido del Alumno",  
    "email": "correo@ejemplo.com",  
    "fecha_inscripcion": "2024-04-10"  
  },  
  {  
    "_id": "alumno_id",  
    "nombre": "Nombre del Alumno",  
    "apellido": "Apellido del Alumno",  
    "email": "correo@ejemplo.com",  
    "fecha_inscripcion": "2024-04-11"  
  }  
]  
}
```

En este modelo, tenemos una colección para los cursos, donde cada documento representa un curso específico. Cada curso tiene un identificador único (`_id`), nombre, descripción, fechas de inicio y fin, la cantidad de cupos disponibles y una lista de alumnos inscritos.

Dentro de la lista de alumnos inscritos, cada alumno se representa como un documento anidado con su propio

identificador único (_id), nombre, apellido, correo electrónico y fecha de inscripción.